

CI/CD による静的サイトと動的 Web アプリの構築レポート

学籍番号: 25G1104

氏名: 沼田 陽

2026 年 7 月 12 日

1 はじめに

本レポートでは、アプリケーションの更新時に自動でデプロイ作業が行われる仕組みである CI/CD (Continuous Integration と Continuous Delivery) を取り上げ、手動でのデプロイと自動デプロイを比較する。あわせて、今回構築した 2 つのサービス、すなわち GitHub Actions による自動デプロイで公開した**静的 Web サイト** (Azure Static Web Apps) と、サーバー側で処理を行う**動的 Web アプリ** (Azure App Service) を題材に、CI/CD と PaaS の利点をまとめる。

2 CI/CD 導入前 (手動でのデプロイ)

CI/CD が用いられていない場合、開発したファイルを開発者自身がサーバーへ配置して、はじめてユーザーが利用できる状態になる。変更のたびに人手で同じ配置作業を繰り返す必要があり、次の課題がある。

- **ヒューマンエラーの発生**：手作業で配置するため、操作ミスや手順の抜けが起こりやすい。
- **手間と時間がかかる**：修正のたびにローカルから同じ内容を送り直す必要があり、更新回数が増えるほど負担が大きくなる。

3 CI/CD 導入後 (自動デプロイ)

CI/CD を導入すると、コードの変更をリポジトリへ反映するだけで、ビルドから公開までが自動で実行される。本課題では、ローカルで編集した HTML / CSS を GitHub の main ブランチへ push するだけで、GitHub Actions が起動し、Azure Static Web Apps へファイルが自動でアップロードされる仕組みを構築した。おおよそ 1 分で公開 URL に反映され、手動でのアップロードは不要となる。これにより、配置作業由来のミスを防げる (**ヒューマンエラーの回避**) とともに、一度仕組みを作れば以降はサーバー配置を意識せず内容の改善に専念できる (**開発への集中**)。

4 今回構築したサービス

4.1 静的 Web サイト (Azure Static Web Apps)

リポジトリ直下に index.html と style.css を置き、.github/workflows/ にデプロイ用 YAML を配置した。main への push を契機に GitHub Actions が実行され、成功時にはログ末尾へ公開 URL が出力される (付録 図 1)。実際に見出しの文字色を変更するなどの修正を push しただけで本番サイトへ即座に反映された (付録 図 2, 図 3)。なお運用中、デプロイ用ワークフローが手動作成分と自動生成成分の 2 本存在して衝突し、push の

たびに一方が失敗していたため、経路を 1 本に整理する重要性も学んだ。

4.2 動的 Web アプリ (Azure App Service, 加点点課題)

追加要件である「Web App Service (動的サイト)」の構築にも成功した。Node.js のフレームワークである **Express** を使い、単なる静的ページの配信ではなく、**リクエストのたびにサーバー側で内容を生成するアプリ**を構築して Azure App Service (PaaS, 無料 F プラン) へデプロイした。具体的には、**現在のサーバー時刻の表示**、メモリ上で数える**アクセスカウンタ**、フォーム入力をサーバー側で受け取って応答を返す**あいさつ機能**を実装した (付録 図 4)。無料枠のため、初回アクセス時はコールドスタートで表示に時間がかかる (重い) 仕様となっている。

5 おわりに

本課題を通じて、手動でのデプロイと自動デプロイの双方を理解した。手作業でのデプロイはミスが起こりやすいが、GitHub Actions による CI/CD を用いることで push するだけで安全かつ短時間に公開できる。さらに、Azure App Service を用いれば、サーバー管理を意識せずに動的な Web アプリまで公開できることを、実際の構築を通して確認できた。

6 付録

6.1 URL

今回構築したサービスの公開 URL は以下の通りである (いずれも 2026 年 7 月 12 日 時点で稼働確認済み)。

静的サイト : <https://orange-stone-0cfe86d00.7.azurestaticapps.net>

動的アプリ : <https://numata-25g1104-express-1833.azurewebsites.net>

※ 静的サイトのドメインには .7. が含まれる。これを省いた URL は 404 になるため注意。

6.2 関連スクリーンショット

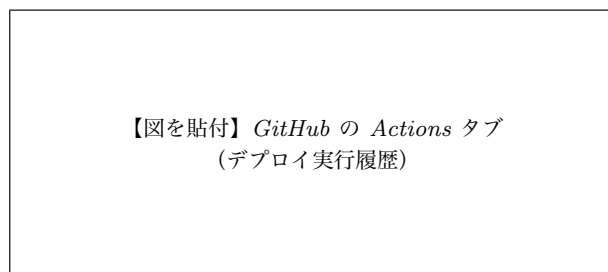


図1 デプロイ画面 (GitHub Actions)

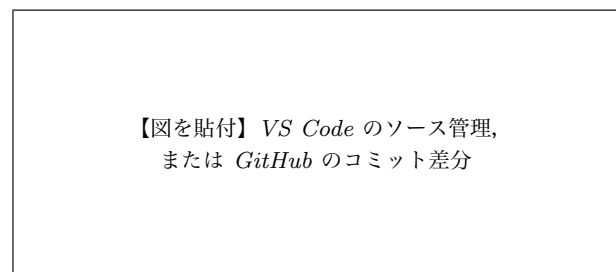


図2 変更画面 (コミット・差分)

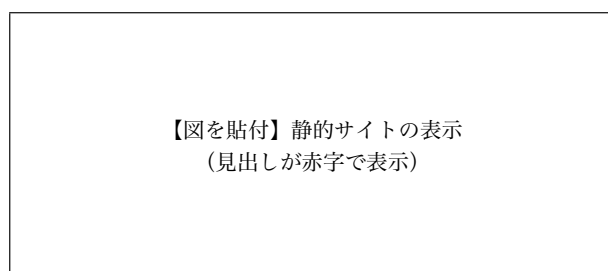


図3 静的サイトの表示 (現在の URL)

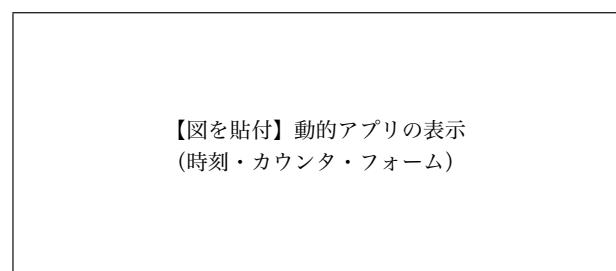


図4 動的アプリの表示 (App Service)